

## Exercise\_04\_KSQL

**1.- Ejecuta desde la terminal los pasos del fichero README asociado al ejercicio. Pega las imágenes de la ejecución de cada uno de los pasos.**

### Paso 1

```
ksql> SET 'auto.offset.reset' = 'earliest';
Successfully changed local property 'auto.offset.reset' to 'earliest'. Use the UNSET command to revert your change.
```

### Paso 2

```
ksql> SHOW TOPICS;
```

| Kafka Topic                                                                                   | Partitions | Partition Replicas |
|-----------------------------------------------------------------------------------------------|------------|--------------------|
| _confluent-command                                                                            | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-actual-group-consumption-rekey                               | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregate-topic-partition-store-changelog                    | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregate-topic-partition-store-repartition                  | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregatedTopicPartitionTableWindows-ONE_MINUTE-changelog    | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregatedTopicPartitionTableWindows-ONE_MINUTE-repartition  | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregatedTopicPartitionTableWindows-THREE_HOURS-changelog   | 1          | 1                  |
| _confluent-controlcenter-7-3-3-1-aggregatedTopicPartitionTableWindows-THREE_HOURS-repartition | 1          | 1                  |

### Paso 3

```
{ "ROWTIME":1744904519141,"ROWKEY":"08320180-33ad-4f62-bce1-5b71ecdf37f4","orderId":"08320180-33ad-4f62-bce1-5b71ecdf37f4","customerId":4,"product":"Enormous Aluminum Plate","amount":74,"price":31.8,"orderedAt":1744706187844}
{"ROWTIME":1744904520159,"ROWKEY":"c9f8f214-8a43-49a1-ba3d-0fcfcd22750b","orderId":"c9f8f214-8a43-49a1-ba3d-0fcfcd22750b","customerId":2,"product":"Rustic Silk Wallet","amount":49,"price":41.2,"orderedAt":1744709953412}
{"ROWTIME":1744904521171,"ROWKEY":"c8c088a9-b24e-44d7-bd3e-f92ed2992b41","orderId":"c8c088a9-b24e-44d7-bd3e-f92ed2992b41","customerId":4,"product":"Sleek Linen Computer","amount":14,"price":81.34,"orderedAt":1744217399585}
{"ROWTIME":1744904522192,"ROWKEY":"36dbc239-21aa-4b43-b708-3e201e015e09","orderId":"36dbc239-21aa-4b43-b708-3e201e015e09","customerId":2,"product":"Ergonomic Leather Wallet","amount":53,"price":8.84,"orderedAt":1744064490424}
{"ROWTIME":1744904523198,"ROWKEY":"09216d44-7a3d-4880-b0bd-f021095db715","orderId":"09216d44-7a3d-4880-b0bd-f021095db715","customerId":3,"product":"Rustic Aluminum Bag","amount":83,"price":82.6,"orderedAt":1744283727096}
{"ROWTIME":1744904524217,"ROWKEY":"6eb058c4-9c18-4ec4-bdb3-d0bdb18f83bb","orderId":"6eb058c4-9c18-4ec4-bdb3-d0bdb18f83bb","customerId":2,"product":"Sleek Steel Knife","amount":23,"price":21.2,"orderedAt":1744411351024}

Press CTRL-C to interrupt
```

### Paso 4

```
ksql> CREATE STREAM orders_stream
> (orderedAt BIGINT,
>  customerId BIGINT,
>  amount BIGINT,
>  price DOUBLE,
>  product VARCHAR)
> WITH (KAFKA_TOPIC='orders',
>       VALUE_FORMAT='JSON',
>       TIMESTAMP='orderedAt');
```

Message

-----

Stream created

-----

ksql>

## Paso 5

```
ksql> SELECT * FROM orders_stream WHERE customerId = 1 EMIT CHANGES;
```

| ROWTIME       | ROWKEY                               | ORDEREDAT     | CUSTOMERID | AMOUNT | PRICE | PRODUCT                 |
|---------------|--------------------------------------|---------------|------------|--------|-------|-------------------------|
| 1744733539316 | 7e90abd0-51ad-468b-816b-252ba1d10e29 | 1744733539316 | 1          | 64     | 30.22 | Durable Bronze Lamp     |
| 1744294431885 | 19bd8f64-f646-42db-911b-606f7b1ef8fa | 1744294431885 | 1          | 37     | 46.73 | Gorgeous Rubber Clock   |
| 1744345102447 | 242ae365-dc8a-4617-bddf-4d191993017a | 1744345102447 | 1          | 35     | 55.41 | Gorgeous Concrete Knife |

Press CTRL-C to interrupt

## Paso 6

```
ksql> SELECT customerId,
> count(*)
> FROM orders_stream
> GROUP BY customerId
> EMIT CHANGES;
```

| CUSTOMERID | KSQL_COL_1 |
|------------|------------|
| 1          | 3          |
| 4          | 15         |
| 3          | 10         |
| 2          | 14         |

Press CTRL-C to interrupt

## Paso 7

```
ksql> CREATE TABLE orders_by_customer AS
> SELECT customerId,
> count(*)
> FROM orders_stream
> GROUP BY customerId
> EMIT CHANGES;
```

Message

Table ORDERS\_BY\_CUSTOMER created and running. Created by query with query ID: CTAS\_ORDERS\_BY\_CUSTOMER\_1

## Paso 8

```
ksql> select * from orders_by_customer emit changes;
```

| ROWTIME       | ROWKEY | CUSTOMERID | KSQL_COL_1 |
|---------------|--------|------------|------------|
| 1744733539316 | 1      | 1          | 3          |
| 1744898767101 | 4      | 4          | 15         |
| 1744901795600 | 3      | 3          | 10         |
| 1744768269160 | 2      | 2          | 14         |

Press CTRL-C to interrupt

**2.- Filtrar por el producto "Awesome Silk Computer". Copia y pega la consulta KSQL ejecutada y el resultado de la ejecución de la consola.**

**Consulta:** SELECT \* FROM orders\_stream WHERE product = 'Awesome Silk Computer' EMIT CHANGES;

```
ksql> SELECT * FROM orders_stream WHERE product = 'Awesome Silk Computer' EMIT CHANGES;
```

Query terminated

```
ksql>
```

No muestra nada, pero la consulta funciona porque filtrando por otros productos sí que los muestra.

**3.- Agrupa los clientes. Copia y pega la consulta KSQL ejecutada y el resultado de la ejecución de la consola.**

**Consulta:**

```
SELECT customerId, COUNT(*) AS order_count
FROM orders_stream
GROUP BY customerId
EMIT CHANGES;
```

```
ksql> SELECT customerId, COUNT(*) AS order_count
>FROM orders_stream
>GROUP BY customerId
>EMIT CHANGES;
+-----+-----+
|CUSTOMERID|ORDER_COUNT|
+-----+-----+
|1|3|
|4|15|
|3|10|
|2|14|
+-----+-----+
^CQuery terminated
```

**4.- Crea una nueva tabla denominada “customer\_totals” que contenga el Identificador del cliente y el total gastado. Copia el código KSQL para crear la tabla. Una vez creada realiza una consulta sobre la misma, copia y pega el código KSQL y el resultado.**

**Consulta crear tabla:**

```
CREATE TABLE customer_totals AS
SELECT customerId, SUM(amount * price) AS total_gastado
FROM orders_stream
GROUP BY customerId
EMIT CHANGES;
```

```
ksql> CREATE TABLE customer_totals AS
>SELECT customerId, SUM(amount * price) AS total_gastado
>FROM orders_stream
>GROUP BY customerId
>EMIT CHANGES;

Message
-----
Table CUSTOMER_TOTALS created and running. Created by query with query ID: CTAS_CUSTOMER_TOTALS_5
-----
```

**Consulta a la tabla:**

```
SELECT * FROM customer_totals EMIT CHANGES;
```

```
ksql> SELECT * FROM customer_totals EMIT CHANGES;
+-----+-----+-----+-----+
|ROWTIME|ROWKEY|CUSTOMERID|TOTAL_GASTADO|
+-----+-----+-----+-----+
|1744733539316|1|1|5602.4400000000005|
|1744898767101|4|4|19745.57|
|1744901795600|3|3|23704.94|
|1744768269160|2|2|32322.309999999998|
+-----+-----+-----+-----+
```